

Objects & Arrays of Objects — JS vs TS vs Go

A quick reference comparing how **single objects** and **arrays of objects** are defined across JavaScript, TypeScript, and Golang.

JavaScript (Dynamic, No Types)

```
javascript

// SINGLE OBJECT
let myCar = {
  Brand: "Tesla",
  Model: "Model 3",
  Year: 2024,
  Color: "Red"
};

// ARRAY OF OBJECTS
let myCars = [
  { Brand: "Tesla", Model: "Model 3", Year: 2024, Color: "Red" },
  { Brand: "Honda", Model: "Civic", Year: 2022, Color: "Blue" },
  { Brand: "Toyota", Model: "Camry", Year: 2023, Color: "Silver" }
];

console.log(myCar);
console.log(myCars);
```

TypeScript (With Types)

```
typescript

// TYPE DEFINITION (like a struct)
type Car = {
  Brand: string;
  Model: string;
  Year: number;
  Color: string;
};

// SINGLE OBJECT with type
let myCar: Car = {
  Brand: "Tesla",
  Model: "Model 3",
  Year: 2024,
  Color: "Red"
};
```

```
};

// ARRAY OF OBJECTS with type
let myCars: Car[] = [
  { Brand: "Tesla", Model: "Model 3", Year: 2024, Color: "Red" },
  { Brand: "Honda", Model: "Civic", Year: 2022, Color: "Blue" },
  { Brand: "Toyota", Model: "Camry", Year: 2023, Color: "Silver" }
];

console.log(myCar);
console.log(myCars);
```

Golang (With Struct)

```
go

package main

import "fmt"

// STRUCT DEFINITION (like type in TS)
type Car struct {
    Brand string
    Model string
    Year  int
    Color string
}

func main() {
    // SINGLE OBJECT (instance of struct)
    myCar := Car{
        Brand: "Tesla",
        Model: "Model 3",
        Year:  2024,
        Color: "Red",
    }

    // ARRAY OF OBJECTS (slice of structs)
    myCars := []Car{
        { Brand: "Tesla", Model: "Model 3", Year: 2024, Color: "Red" },
        { Brand: "Honda", Model: "Civic", Year: 2022, Color: "Blue" },
        { Brand: "Toyota", Model: "Camry", Year: 2023, Color: "Silver" },
    }
```

```
fmt.Println(myCar)
fmt.Println(myCars)
}
```

Comparison Table

Feature	JavaScript	TypeScript	Golang
Type definition	✗ None (dynamic)	type Car = {...}	type Car struct {...}
Single object	let x = {...}	let x: Car = {...}	x := Car{...}
Array of objects	let arr = [...]	let arr: Car[] = [...]	arr := []Car{{...}}
Type safety	✗ No	✓ Yes	✓ Yes
Compile-time checks	✗ No	✓ Yes	✓ Yes

Output (All Three Produce Similar Results)
Single Object:

```
{ Brand: 'Tesla', Model: 'Model 3', Year: 2024, Color: 'Red' }
```

Array of Objects:

```
[
  { Brand: 'Tesla', Model: 'Model 3', Year: 2024, Color: 'Red' },
  { Brand: 'Honda', Model: 'Civic', Year: 2022, Color: 'Blue' },
  { Brand: 'Toyota', Model: 'Camry', Year: 2023, Color: 'Silver' }
]
```

Key Takeaways

- **JavaScript** — Flexible but no type safety; great for quick scripts.
- **TypeScript** — Adds a `type` alias to enforce structure at compile time; ideal for large JS codebases.
- **Golang** — Uses a `struct` for strict typing; compiled and performant, best for systems/backend work.